Rheinland-Pfälzische
Technische Universität
Kaiserslautern
Landau

Optimizing Error Messages: The Impact of Optimized Error Messages in Teaching F#

Bachelor's Thesis

by

Jakob Meyerhöfer

March 27, 2025

University of Kaiserslautern-Landau
Department of Computer Science
67663 Kaiserslautern
Germany

Examiner: Prof. Dr. Ralf Hinze
Sebastian Schloßer, M.Sc. (Sebastian Schweizer)

Declaration of Authorship

I hereby declare that the work I have submitted with the title „Optimizing Error Messages: The Impact of Optimized Error Messages in Teaching F#“ was independently written by me. I have fully disclosed all sources and aids used and have identified as quotations all passages of the work — including tables and figures — taken verbatim or in substance from other works or the internet, indicating the respective sources.

Kaiserslautern, on 27.3.2025

Jakob Meyerhöfer

Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Summary

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Contents

1. Introduction **1**
- 1.1. Research Problem and Objectives . . . 2
- 1.2. Research Questions . . . 3
- 1.3. Thesis Structure . . . 3

2. Related Work **5**
- 2.1. Challenges for Novice Programmers . . . 5
- 2.2. Role of Error Messages in Learning . . . 6
- 2.3. Existing Approaches to Improving Error Messages . . . 6
- 2.4. Educational Feedback Theories . . . 7

3. Analysis of Previous Semester Data **9**
- 3.1. Data Collection and Methodology . . . 9
- 3.2. Compilation Success Rates . . . 10
- 3.3. Test Pass Rates . . . 10
- 3.4. Insights and Patterns . . . 12

4. Problem Analysis: Issues with Existing Error Messages **19**
- 4.1. Examples of Current Error Messages . . . 19
  - 4.1.1. Example 1 — `digitSum` . . . 19
  - 4.1.2. Example 2 — `sortedDigits` . . . 23
- 4.2. Information Gaps in Error Outputs . . . 26
- 4.3. Challenges for Novices in Deriving Test Information . . . 27

5. Assertify: A Library for Improving Error Messages **29**
- 5.1. Objectives and Design of Assertify . . . 29
- 5.2. Implementation Details and Wrappers . . . 29
- 5.3. Error Message Improvements (Before and After Comparisons) . . . 30
  - 5.3.1. Example 1 — `digitSum` . . . 31
  - 5.3.2. Example 2 — `sortedDigits` . . . 32
- 5.4. Usability and Design Considerations . . . 32

6. TODO: Further Methodology (unclear what comes next) **35**

7. Evaluation and Results **37**
- 7.1. Evaluation Methodology . . . 37
- 7.2. Metrics for Success . . . 37
- 7.3. Quantitative Analysis . . . 37
- 7.4. Qualitative Insights . . . 37
- 7.5. Comparison to Previous Results . . . 38

8. Conclusion **39**
8.1. Summary of Contributions 39
8.2. Key Findings 39
8.3. Interpretation of Results 39
8.4. Strengths and Limitations of Approach 39
8.5. Recommendations for Future Development 40

List of Figures **41**

List of Listings **43**

List of Tables **45**

A. My Code **47**

B. My Ideas **49**

Bibliography **51**

1. Introduction

All programmers have to start at the beginning: What even is programming? What do you need to start writing code? What are variables/methods/etc. and how do you use them? Then there are certain key concepts/algorithms and further down the road data structures like maps, sets and so on. But that is a long way to go from learning the very basics to independently writing your own programs in the end, including the implementations and the according unit tests to verify said implementations.

Understanding and interpreting error messages is a fundamental challenge for novice programmers taking their first steps. Error messages serve as an essential form of feedback, helping programmers diagnose issues in their code. However, standard error messages, particularly those generated by compilers and testing frameworks, are often cryptic, overly technical, and difficult for beginners to decipher. Especially when students are provided with templates that contain tests and the goal is to implement various methods or data structures that can then be tested via the provided tests. They are often not particularly familiar with unit testing and are now confronted with stacktraces and various formats of error output depending on the programming language and testing framework.

In F# there are various namespaces or testing frameworks available. A commonly used namespace is Microsoft.VisualStudio.TestTools.UnitTesting. For example by using the `Assert.AreEqual` method one might get the following output:

```
path\to\Tests.fs(68): error TESTERROR:
    a) digitSum Random (75ms): Error Message: Test method Tests+Tests.a) digitSum Random threw exception:
    System.Exception: Falsifiable, after 1 test (1 shrink) (StdGen (866870754, 297439050)):
    Original:
    9N
    Shrunk:
    0N
    with exception:
    Microsoft.VisualStudio.TestTools.UnitTesting.AssertFailedException: Assert.AreEqual failed. Expected:<0>. Actual:<1>.
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.ThrowAssertFailed(String assertionName, String message) in /_/src/TestFramework/TestFramework/Assertions/Assert.cs:line 60
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T](T expected, T actual, IEqualityComparer`1 comparer, String message, Object[] parameters) in /_/src/TestFramework/TestFramework/Assertions/Assert.AreEqual.cs:line 201
```

```
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T]
        ↪  (T expected, T actual) in /_/src/TestFramework/TestFramework/
        ↪  Assertions/Assert.AreEqual.cs:line 34
        at Tests.a) digitSum Random@69.Invoke(Nat n) in path\to\Tests.fs:
        ↪  line 69
        at FsCheck.Testable.evaluate[a,b](FSharpFunc`2 body, a a)

    Stack Trace:
        at Microsoft.FSharp.Core.PrintfModule.PrintFormatToStringThenFail@
        ↪  1448.Invoke(String message)
        at <StartupCode$FsCheck>.$Runner.get_throwingRunner@382.FsCheck.
        ↪  IRunner.OnFinished(String name, TestResult testResult)
        at FsCheck.Runner.runner[a](Config config, a prop)
        at FsCheck.Runner.check[a](Config config, a p)
        at FsCheck.Check.One[Testable](Config config, Testable property)
        at Tests.Tests.a) digitSum Random() in path\to\Tests.fs:line 68
        at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
        ↪  arguments, Signature sig, Boolean isConstructor)
        at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj
        ↪  , BindingFlags invokeAttr)
```

Listing 1.1. Example Ouput of `Assert.AreEqual`

For advanced programmers this does not look too cryptic or misleading, because they are used to extract the necessary information they need from this. See section 4 on page 19 on issues with error messages and how to extract this key information. Working as a tutor for a first semester lecture, I frequently observe students struggling with these messages, unsure of what went wrong and how to fix their code. Students with none to some programming experience alike. Instead of guiding them toward a solution, poorly structured error messages often lead to frustration and trial-and-error debugging. This hinders the learning process and increases the time required to grasp key programming concepts.

To address this issue, this thesis explores how improving error messages can enhance the learning experience for novice programmers. By providing clearer, more structured feedback, the aim is to help students understand their mistakes more effectively, ultimately leading to a more efficient and engaging learning process.

1.1. Research Problem and Objectives

The primary problem this thesis addresses is the difficulty novice programmers face when interpreting error messages. Standard compiler and testing framework messages often assume prior knowledge, making them inaccessible to beginners. This can lead to confusion, incorrect assumptions about the problem, and inefficient debugging strategies. The objectives of this research are as follows:

- Analyze the impact of traditional error messages on the learning process.
- Develop a modified approach that enhances the clarity and usefulness of error messages.
- Evaluate whether improved error messages lead to faster learning and better debugging strategies among students.

To achieve these objectives, I have developed a test class that extends standard error reporting by adding a compact summary of test results. This modification aims to provide clearer feedback, helping students identify and resolve issues more efficiently.

1.2. Research Questions

This thesis seeks to answer the following key research questions:

- How do traditional error messages impact the learning process of novice programmers?
- What specific issues do students encounter when interpreting standard error messages?
- Can modified error messages improve comprehension and debugging efficiency?
- How does structured feedback affect students' ability to correct their mistakes?

By addressing these questions, this research aims to provide insights into the effectiveness of improved error reporting and its potential role in programming education.

1.3. Thesis Structure

This thesis is structured as follows:

- **Chapter 2: Related Work** – Reviews existing research on challenges faced by novice programmers, the role of error messages in learning, and previous attempts at improving error feedback.
- **Chapter 3: Analysis of Previous Semester Data** – Presents an analysis of past student submissions, including compilation success rates and test pass rates, to identify common issues.
- **Chapter 4: Problem Analysis** – Examines specific problems with existing error messages, including real-world examples and gaps in the information they provide.
- **Chapter 5: Assertify — A Library for Improving Error Messages** – Introduces and explains the implementation of Assertify, a tool designed to enhance error feedback.
- **Chapter 6: Further Methodology (VS Code Extension?)** – TODO
- **Chapter 7: Evaluation and Results** – Presents an analysis of the impact of modified error messages, including both quantitative and qualitative findings.

- **Chapter 8: Conclusion** – Summarizes key findings, discusses their implications, and provides recommendations for future work.

This structure ensures a logical flow, moving from problem identification to solution development and evaluation, ultimately contributing to a better understanding of how error messages affect novice programmers.

2. Related Work

In this chapter, I will elaborate on findings from related work, primarily focusing on two papers: The first is by Becker et al. [2] that investigates the helpfulness compiler error messages and the second paper about whether or not programmers actually read error messages by Barik et al. [1], which is also referenced in the first one. Their work provides a comprehensive analysis of the challenges faced by novice programmers when dealing with compiler error messages and suggests various ways to improve these messages to enhance learning outcomes. According to Barik et al. [1] programmers do in fact read error messages. Novices interact with them just as much by trying to decrypt them, researching what they mean and how that corresponds to their implementations. At the worst students simply copy-paste their code and the according error message into any available AI and ask it to fix the problem.

2.1. Challenges for Novice Programmers

Barik et al. [1] names important findings in their paper that pose challenges for programmers, novices and further progressed programmers alike:

- Error messages are comparably difficult to read as reading plain source code.
- The difficulty in understanding error messages is a significant predictor of participants' performance on tasks.
- And lastly participants' spend $13 - 25\%$ of their total assignment reading error messages. That is a considerable portion of that time, considering that this means of 4 hours coding, 31 to 60 minutes are speant with only reading error messages.

One of the significant barriers to learning programming is the difficulty that novices face in interpreting compiler error messages. Becker et al. highlight that compiler messages are often cryptic, overly technical, and not designed with educational usability in mind [2]. Many students struggle with understanding the cause of errors, leading to frustration, increased cognitive load, and decreased motivation.

Research has shown that compiler error messages can act as "barriers to progress" [2], causing students to spend excessive time debugging trivial syntax issues rather than focusing on problem-solving and program design. Additionally, misconceptions about error messages often lead to incorrect debugging strategies, which further hinder learning progress.

2.2. Role of Error Messages in Learning

Compiler error messages play a crucial role in the debugging and learning process. Becker et al. point out that error messages are one of the primary feedback mechanisms for students learning to program.

When designed effectively, these messages can facilitate learning by providing clear guidance on how to correct mistakes. However, standard compiler messages are often ambiguous or misleading, requiring students to rely on trial and error or seek external help to resolve issues. Research has demonstrated that students who repeatedly encounter confusing error messages may develop negative attitudes toward programming, reducing persistence and motivation.

Barik et al. [1] name internal and external factors, that affect students:

- **Internal factors:** These refer to the ways students personally react to and interact with error messages. Research shows that students often find compiler error messages frustrating, intimidating, and confusing, which can negatively impact their confidence. As a result, they tend to distrust error messages, even though they do read them. Furthermore, error messages influence students' behavior when programming, although this is something many instructors have already observed firsthand.
- **External factors:** These involve external influences on how students experience error messages. Studies indicate that the way students interact with error messages can be linked to traditional academic success metrics and is an observable aspect of their programming behavior. Additionally, dealing with error messages can consume valuable teaching time and resources. Moreover, the programming environment itself plays a crucial role in shaping a student's experience with error messages, as different environments provide varying levels of clarity and support.

2.3. Existing Approaches to Improving Error Messages

Several approaches have been proposed to improve the effectiveness of compiler error messages. Becker et al. categorize these improvements into three main areas:

- **Enhanced Error Messages:** Modifying existing messages to make them more understandable for novices. This includes providing clearer wording, suggestions for corrections, and context-sensitive hints.
- **Integrated Debugging Tools:** Some educational programming environments incorporate intelligent tutoring systems that analyze student errors and provide targeted feedback.
- **Community-Driven Solutions:** Platforms like Stack Overflow and online programming forums allow students to find explanations and solutions for common compiler errors. Some IDEs now integrate crowdsourced solutions to improve debugging assistance.

Despite these advancements, Becker et al. [2] note that many of these solutions have limitations, not only that but in many programming languages it is still the

case that: Firstly, a isolated error may cause different error messages depending on the context and furthermore, the same error messages can be produce by entirely diffent and definite errors. While enhanced messages and debugging tools can improve comprehension, they may still fail to address deeper misconceptions about programming concepts. Additionally, community-driven solutions are not always reliable, as the quality of responses varies.

2.4. Educational Feedback Theories

The importance of feedback in education has been widely studied in pedagogical research, and several theories can be applied to compiler error messages. Becker et al. discuss how principles from feedback theory, such as formative assessment and scaffolding, can be leveraged to improve error messages. Effective feedback should be timely, specific, and actionable, enabling students to correct their mistakes and understand underlying concepts.

Furthermore, research suggests that error messages should be designed to support a growth mindset. Encouraging messages that normalize errors as part of the learning process can help students develop resilience and persistence in programming. Some studies advocate for adaptive error messages that adjust based on a student's experience level, providing more detailed explanations for beginners while offering concise feedback for more advanced learners.

3. Analysis of Previous Semester Data

In this chapter I will go over the data I analyzed: What data, how it was collected and so on.

3.1. Data Collection and Methodology

The data set I analyzed was collected during winter semester 2023/2024 in the module Concepts of Programming. The data was anonymized and forwarded to me by my supervisor. There were 220 students in total submitting their files individually. During this semester the students had to complete 12 sheets with 16 tasks that are relevant. *Relevant* in this case means that these were coding tasks with unit tests to automatically detect the correctness of the submission. The students were provided with a template project containing the following:

- A file called `Mini.fs` – a small library of useful methods and basic types that are used throughout all of the assignments. For instance `Mini.fs` simplifies the user input and output to shift the focus away from all possible edge cases of I/O this early on. Mostly used is the type `Nat`, describing a natural number including zero ($\mathbb{N}_0$), written as a non-negative integer with postfix '`N`': `0N`, `1N`, ..., `1234N`, etc.
- Further assignment-specific types (e.g. type definition of a linked list and the individual nodes)
- All relevant methods with their method signature and a placeholder `failwith "TODO"` for the body

The unit tests normally consist of:

- Example tests that verify edge cases and specific method calls given in the problem definition. If the goal is to implement a method, the problem definition lists multiple method calls that emphasize the functioning of said method.
- Random tests that verify an implementation against random input.

They did not have access to these tests, but they were able to execute them on the same platform where they submit their solutions (ExClaim). They do not have to execute any commands or change working directories in a terminal, but can rather verify their solutions individually by a simple button press. Through this in particular I had access to all submissions for a single assignment – in other words the history – to analyze the progression.

To illustrate, consider a method that returns the minimum of 3 natural numbers. The template for this method might look as follows:

Template

```
let minimumOfThreeNumbers (a: Nat) (b: Nat) (c: Nat): Nat =
    failwith "TODO"
```

The method is implemented with the placeholder `failwith "TODO"` to ensure compilation even if not all methods are implemented so that they are still able to test their solution at any point. The students are supposed to replace this `failwith "TODO"` with their own implementation of that method. The problem definition then might list:

```
minimumOfThreeNumbers 1N 2N 3N        minimumOfThreeNumbers 0N 0N 0N
minimumOfThreeNumbers 2N 3N 1N        minimumOfThreeNumbers 2N 2N 3N
```

to illustrate that neither the order of the input nor duplicates influence the end result. So exactly these method calls would be used for the example tests.

Sample Solution

```
let minimumOfThreeNumbers (a: Nat) (b: Nat) (c: Nat): Nat =
    min (min a b) c
```

In this sample solution the placeholder was replaced by a compact solution for the assignment. Student submissions may very well contain `match`-statements, `if-else`-statements or similar structures, which are also considered to be valid solutions.

The given data set consists of:

- All project templates including the unit tests
- All uploaded files uniquely identified with timestamps and group/team IDs
- Timestamps of executed tests
- Timestamps of deletions or updates of files by the students

With this data I reconstructed every single submission snapshot, i.e. which files were uploaded when tests were executed. Then I used Docker containers to collect the results of the build (Command: `dotnet build`) and test process (Command: `dotnet test`) for further analysis. The results of that will be elaborated in the following sections.

3.2. Compilation Success Rates

In total there are 12812 submission snapshots, i.e. instances where students uploaded one or multiple files to test their implementation. Of all snapshots 6829 (53.30%) are compilable, while the other 5983 (46.70%) snapshots failed to build and therefore no test results can be determined. For graphical representation see figure 3.1 on page 11.

3.3. Test Pass Rates

An interesting measure is to compare the first compilable snapshot with the last to track the overall progress. Through that following semesters with access to op-

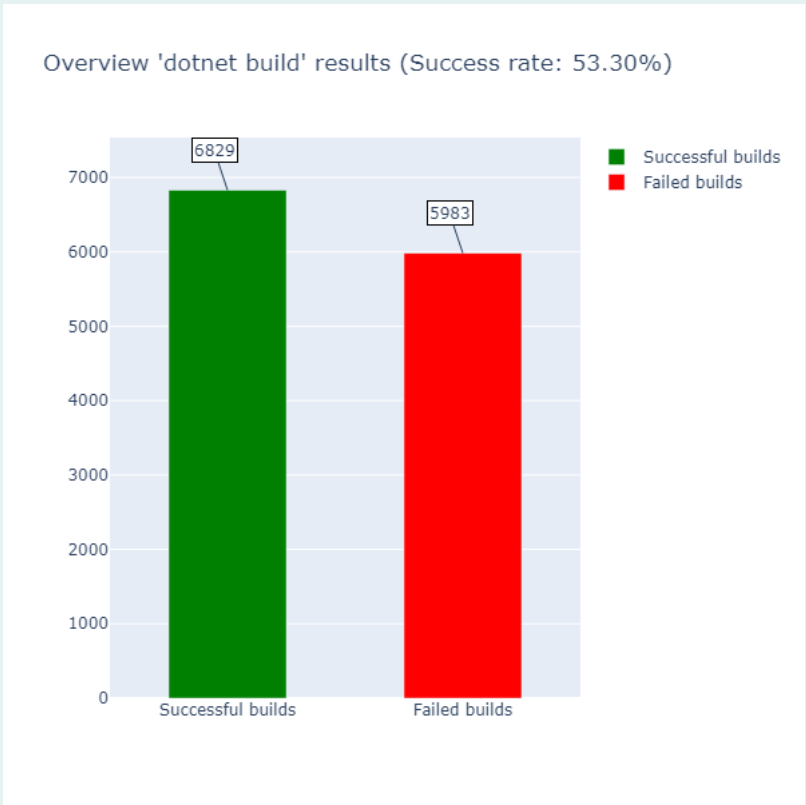

Figure 3.1. Build Stability Analysis: Success and failure counts for .NET builds, with overall success percentage.

timized test results and feedback can be compared to this one to look for possible improvements. In the winter semester 2023/2024 the first compilable snapshot achieved 57.53% passed tests on average, while the last one achieved 87.99% passed tests on average. In the following figure the percentage of passed tests of the first and last compilable snapshot are given for each assignment indicidually. For graphical representation see figure 3.2 on page 12.

Figure 3.2. Submission Performance Analysis: Success rate of initial and final submissions across different exercises.

3.4. Insights and Patterns

For all of the failed submissions, i.e. the snapshots that were unable to build successfully, I looked at all occurring error codes. In table 3.1 (Place 1 to 5 of the Top 10) on page 15 and table 3.2 (Place 6 to 10 of the Top 10) on page 16 is the top 10 given with a percentage on how much that error code makes up in relation to all errors, as well as a short code snippet that triggers that error code. In total 94 different error codes occurred, the distribution is show in figure 3.3 on page 17. Now I will go briefly over the two most common errors, because they alone make up over 50% of all errors:

- **FS0001:** Type mismatch
- **FS0058:** Offside lines

Every third error is a type mismatch (error code: FS0001), which can occur on multiple instances, when in one way or a another type `A` is expected, but it given is a value of type `B` and it is not possible to implicitly cast type `B` to type `A`.
A few instances of that being:

- **Assigning variables/attributes:**

```
let x: int = "Hello"
-----------------------------------------------------------------
error FS0001:  This expression was expected to have type 'int' but
here has type 'string'.
```

```
type A = {X: int}
let a: A = {X = "Hello"}
-----------------------------------------------------------------
error FS0001:  This expression was expected to have type 'int' but
here has type 'string'.
```

- **Returning values:**

```
let f (): bool = 0
-----------------------------------------------------------------
error FS0001:  This expression was expected to have type 'bool' but
here has type 'int'.
```

```
let f (n: float): int = n
-----------------------------------------------------------------
error FS0001:  This expression was expected to have type 'int' but
here has type 'float'.
```

While this would be ok `let f (n: int): float = n`.

- **Inconsistency when branching or matching:**

```
let foo x = if x <> 0 then "Hello" else 4711
-----------------------------------------------------------------
error FS0001:  All branches of an 'if' expression must return values
implicitly convertible to the type of the first branch, which here is
'string'.  This branch returns a value of type 'int'.
```

```
match 1.0 with | 1.0 -> "Hello" | _ -> 0.0
-----------------------------------------------------------------
error FS0001:  All branches of a pattern match expression must return
values implicitly convertible to the type of the first branch, which
here is 'string'.  This branch returns a value of type 'float'.
```

- **Calling functions wrong:**

```
let multiply x y = x * y
multiply (1, 2)
-----------------------------------------------------------------
error FS0001:  This expression was expected to have type 'int' but
here has type 'int * int'.
```

Correct would be `multiply 1 2`.

- **Operator precedence, i.e. the order of how operators are applied:**

```
let s = printf "What is your name? "; System.Console.ReadLine ()
printfn "Your name is: " + s
-----------------------------------------------------------------
error FS0001:  The type 'string' does not match the type 'unit'.
```

Correct would be `printfn ("Your name is: " + s)`.

Every fifth error has to do with offside lines/indentation (error code: FS0058) which means that some structured construct such as `if`/`then`/`else`, `try`/`with`, `let`, etc. introduced an offside line and the indentation is not the same at any line of that block or there is no indentation to begin with. Examples include when an implementation of a method is places on a new line after the method signature and there is no offset of the code or inconsistent offset.

```
let foo (x: int) =
if x <> 0 then "A" else "B"
-----------------------------------------------------------------------
error FS0058:  Unexpected syntax or possible incorrect indentation:  this
token is offside of context started at position (1:1).  Try indenting this
further.  To continue using non-conforming indentation, pass the
'-strict-indentation-' flag to the compiler, or set the language version to
F# 7.
```

```
match 1.0 with
 | 1.0 -> "A"
| _ -> "B"
-----------------------------------------------------------------------
error FS0058:  The '|' tokens separating rules of this pattern match are
misaligned by one column.  Consider realigning your code or using further
indentation.
```

To illustrate an actual example of that is the method to compute the minimum of three numbers `minimumOfThreeNumbers`:

```
let minimumOfThreeNumbers (a: Nat) (b: Nat) (c: Nat): Nat =
    failwith "TODO"
```

Here is an example of what a random submission for that assignment looks like:

```
let minimumOfThreeNumbers (a: Nat) (b: Nat) (c: Nat): Nat =
if a <= b then
    if a <= c then a else c
else
    if b <= c then b else c
-----------------------------------------------------------------------
error FS0058:  Unexpected syntax or possible incorrect indentation:  this
token is offside of context started at position (7:1).  Try indenting this
further.To continue using non-conforming indentation, pass the
'-strict-indentation-' flag to the compiler, or set the language version to
F# 7.
```

which is algorithmically not wrong, in fact indenting the code correctly like

```
let minimumOfThreeNumbers (a: Nat) (b: Nat) (c: Nat): Nat =
    if a <= b then
        if a <= c then a else c
    else
        if b <= c then b else c
```

will not only compile but implement the method to have the desired purpose.

Pos	%	Code example with resulting error code/message
1st	35.35%	`let x: int = true` `error FS0001: This expression was expected to have type 'int' but here has type 'bool'.`

2nd	20.69%	`let f x = if x then 1 else`
		`-1`
		`error FS0058: Unexpected syntax or possible incorrect indentation: this token is offside of context started at position (1:11). Try indenting this further. To continue using non-conforming indentation, pass the '-strict-indentation-' flag to the compiler, or set the language version to F# 7.`

3rd	15.28%	`let add a b =`
		`  let result = a + b`
		`error FS0588: The block following this 'let' is unfinished. Every code block is an expression and must have a result. 'let' cannot be the final code element in a block. Consider giving this block an explicit result.`

4th	9.42%	`printfn "%d" List.min([1; 2])` `error FS0597: Successive arguments should be separated by spaces or tupled, and arguments involving function or method applications should be parenthesized.`
5th	4.45%	`if 1 < 2 then 1 else` `error FS3524: Expecting expression.`

Table 3.1. The top 10 of F# error codes in all submissions (1st – 5th)

Pos	%	Code example with resulting error message

6th	2.70%	`type Ex = | A of int * int`
		`match A(1,2) with A (, y) -> y`
		`error FS3546: Expecting pattern.`

7th	1.82%	`type Ex<'a,'b> = {A:'a; B:'b}`
		`let ex: Ex<int> = {A=1; B=2}`
		`error FS0033: The type 'FSI_0002.Ex<_,_>' expects 2 type argument(s) but is given 1.`

8th	1.66%	`type Ex = {A: int; B: int}`
		`let ex = Ex`
		`error FS0800: Invalid use of a type name.`

9th	1.34%	`let add a b = a +` `error FS3156: Unexpected token '+' or incomplete expression.`

10th	0.68%	`type Ex = | A of int * int`
		`match A (1, 2) with A x y -> x`
		`error FS0727: This union case expects 2 arguments in tupled form, but was given 0.`
		`The missing field arguments may be any of:`
		`  int`
		`  int`

other	6.61%	—

Table 3.2. The top 10 of F# error codes in all submissions (6th – 10th)

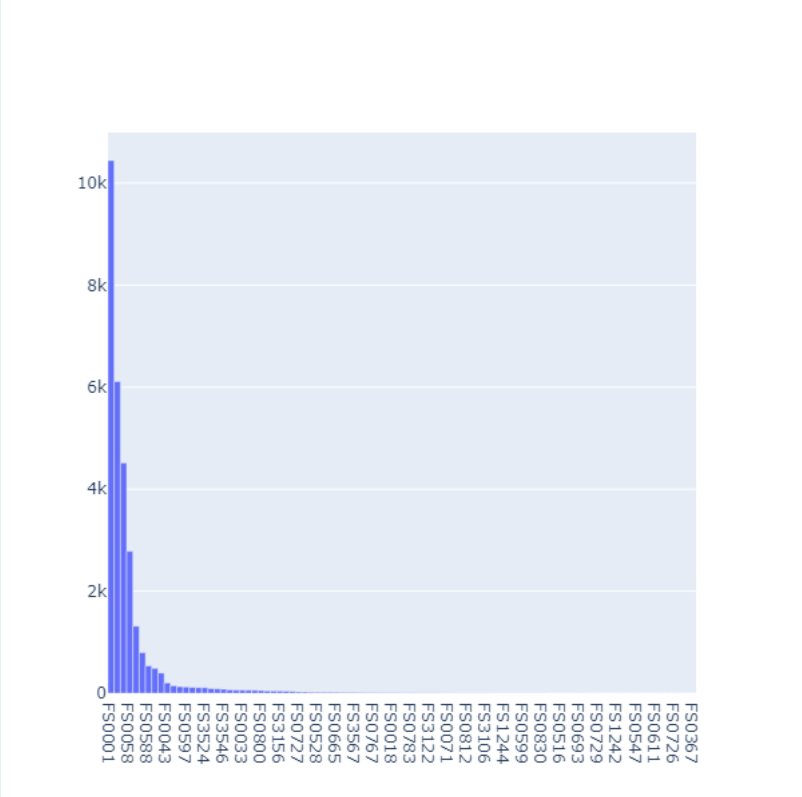

Figure 3.3. Error Code Analysis: Distribution and frequency of F#error codes across all exercises and submissions.

4. Problem Analysis: Issues with Existing Error Messages

In this chapter I will explain known issues for novices when working with error messages.

4.1. Examples of Current Error Messages

Error messages often require the programmer to not only know what was actually tested but how it was tested as well. For instance are characteristics of data structures sometimes tested multiple times between altering the stored data (e.g. adding or removing elements). To then comprehend the error message of such a test fully would require one to know every alteration done before the test/assertion failed to reproduce and retrace the steps and by that find the flaw in their implementation.

Furthermore are some error messages formatted in a manner such that essential data for novices is broken up and/or hidden. In particular does `Assert.AreEqual` not show which method/expression was tested when the error occurred and rather focues on the input. Without including the name of the tested method in the signature of the test method a student would not be able to connect the test results with any particular part of their code which renders the error message basically useless. Even if the name is included you still have to connect the separate pieces to reconstruct the explicit method call that caused this error message.

4.1.1. Example 1 — `digitSum`

Implement a method `digitSum: Nat -> Nat`, that takes a natural number `n` as input and returns its digit sum. Examples:

```
digitSum 0N = 0N
digitSum 123N = 6N
digitSum 1234N = 10N
digitSum 4711N = 13N
```

```
let rec digitSum (n: Nat): Nat =
    if n = 0N then
        1N
    else
        (n % 10N) + digitSum (n / 10N)
```

Listing 4.1. Student submission for `digitSum`

```
let rec digitSumSolution (n: Nat): Nat =
    if n = 0N then
        0N
    else
        (n % 10N) + digitSumSolution (n / 10N)
```

Listing 4.2. Sample solution for `digitSum`

Example Tests — digitSum

```
    [<TestMethod>] [<Timeout(1000)>]
    member _.``a) digitSum Examples`` (): unit =
        test <@ digitSum 123N  = 6N @>
        test <@ digitSum 1234N = 10N @>
        test <@ digitSum 42N   = 6N @>
        test <@ digitSum 105N  = 6N @>
        test <@ digitSum 0N    = 0N @>
        test <@ digitSum 4711N = 13N @>
```

Listing 4.3. Example tests for digitSum

```
a) digitSum Examples (388ms): Error Message: Test method Tests+Tests.a)
↪  digitSum Examples threw exception:
    Swensen.Unquote.AssertionFailedException: Test failed:

    StudentSubmission.digitSum 123N = 6N
    StudentSubmission.digitSum 123N = 6N
    7N = 6N
    false

    Stack Trace:
        at Tests.Tests.a) digitSum Examples() in path\to\Tests.fs:line 57
        at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
        ↪  arguments, Signature sig, Boolean isConstructor)
        at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj
        ↪  , BindingFlags invokeAttr)
```

Listing 4.4. Output of examples tests for digitSum

Key Information Example Tests — digitSum

Tested expression	digitSum 123N
Expected result	6N
Actual result	7N

Table 4.1. Key Information Example Tests — digitSum

Random Tests — digitSum

```
    [<TestMethod>] [<Timeout(5000)>]
    member _.``a) digitSum Random`` (): unit =
        Check.One (
            { Config.QuickThrowOnFailure with EndSize = 1000 },
            fun (n: Nat) -> Assert.AreEqual<Nat> (digitSumSolution n,
            ↪  digitSum n)
        )
```

Listing 4.5. Random tests for digitSum

```
a) digitSum Random (74ms): Error Message: Test method Tests+Tests.a)
↪  digitSum Random threw exception:
    System.Exception: Falsifiable, after 1 test (1 shrink) (StdGen (
    ↪  1765833463, 297438798)):
    Original:
    11N
    Shrunk:
    0N
    with exception:
    Microsoft.VisualStudio.TestTools.UnitTesting.AssertFailedException:
    ↪  Assert.AreEqual failed. Expected:<0>. Actual:<1>.
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.
        ↪  ThrowAssertFailed(String assertionName, String message) in /_/
        ↪  src/TestFramework/TestFramework/Assertions/Assert.cs:line 60
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T]
        ↪  (T expected, T actual, IEqualityComparer`1 comparer, String
        ↪  message, Object[] parameters) in /_/src/TestFramework/
        ↪  TestFramework/Assertions/Assert.AreEqual.cs:line 201
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T]
        ↪  (T expected, T actual) in /_/src/TestFramework/TestFramework/
        ↪  Assertions/Assert.AreEqual.cs:line 34
        at Tests.a) digitSum Random@69.Invoke(Nat n) in path\to\Tests.fs:
        ↪  line 69
        at FsCheck.Testable.evaluate[a,b](FSharpFunc`2 body, a a)

    Stack Trace:
        at Microsoft.FSharp.Core.PrintfModule.PrintFormatToStringThenFail@
        ↪  1448.Invoke(String message)
        at <StartupCode$FsCheck>.$Runner.get_throwingRunner@382.FsCheck.
        ↪  IRunner.OnFinished(String name, TestResult testResult)
        at FsCheck.Runner.runner[a](Config config, a prop)
        at FsCheck.Runner.check[a](Config config, a p)
        at FsCheck.Check.One[Testable](Config config, Testable property)
        at Tests.Tests.a) digitSum Random() in path\to\Tests.fs:line 68
        at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
        ↪  arguments, Signature sig, Boolean isConstructor)
        at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj
        ↪  , BindingFlags invokeAttr)
```

Listing 4.6. Output of random tests for digitSum

Key Information Random Tests - digitSum

Tested expression	`a) digitSum Random` / `Shrunk: 0N` } `digitSum 0N`
Expected result	`Expected:<0>` $\Longrightarrow$ `0` (actually `0N`)
Actual result	`Actual:<1>` $\Longrightarrow$ `1` (actually `1N`)

Table 4.2. Title

4.1.2. Example 2 — `sortedDigits`

Implement a method `sortedDigits: Nat -> bool`, that takes a natural number `n` as input and returns `true` if the digits are in ascending order and `false` otherwise. Examples:

```
sortedDigits 0N = true          sortedDigits 1111N = true
sortedDigits 5N = true          sortedDigits 1101N = false
sortedDigits 159N = true        sortedDigits 543N = false
```

```
let rec sortedDigits (n: Nat): bool =
    if n = 0N then
        true
    else
        let frontSorted: bool = sortedDigits (n / 10N)
        let lastDigit: Nat = n % 10N
        let secondLastDigit: Nat = (n / 10N) % 10N
        frontSorted && secondLastDigit < lastDigit
```

Listing 4.7. Student submission for `sortedDigits`

```
let rec sortedDigitsSolution (n: Nat): bool =
    if n = 0N then
        true
    else
        let frontSorted: bool = sortedDigitsSolution (n / 10N)
        let lastDigit: Nat = n % 10N
        let secondLastDigit: Nat = (n / 10N) % 10N
        frontSorted && secondLastDigit <= lastDigit
```

Listing 4.8. Sample solution for `sortedDigits`

Example Tests — `sortedDigits`

```
    [<TestMethod>] [<Timeout(1000)>]
    member _.``b) sortedDigits Examples`` (): unit =
        test <@ sortedDigits 0N    = true @>
        test <@ sortedDigits 5N    = true @>
        test <@ sortedDigits 159N  = true @>
        test <@ sortedDigits 1111N = true @>
        test <@ sortedDigits 42N   = false @>
        test <@ sortedDigits 543N  = false @>
        test <@ sortedDigits 1101N = false @>
```

Listing 4.9. Example tests for `sortedDigits`

```
b) sortedDigits Examples (4ms): Error Message: Test method Tests+Tests.b)
↪  sortedDigits Examples threw exception:
   Swensen.Unquote.AssertionFailedException: Test failed:

   StudentSubmission.sortedDigits 1111N = true
   StudentSubmission.sortedDigits 1111N = true
   false = true
   false

   Stack Trace:
        at Tests.Tests.b) sortedDigits Examples() in path\to\Tests.fs:line
        ↪   80
        at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
        ↪  arguments, Signature sig, Boolean isConstructor)
          at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object
          ↪  obj, BindingFlags invokeAttr)
```

Listing 4.10. Output of examples tests for `sortedDigits`

Key Information Example Tests — `sortedDigits`

Tested expression	`sortedDigits 123N`
Expected result	`true`
Actual result	`false`

Table 4.3. Title

Random Tests — `sortedDigits`

```
    [<TestMethod>] [<Timeout(5000)>]
    member _.``b) sortedDigits Random`` (): unit =
        Check.One (
            { Config.QuickThrowOnFailure with EndSize = 1000 },
            fun (n: Nat) -> Assert.AreEqual<bool> (sortedDigitsSolution n,
            ↪   sortedDigits n)
        )
```

Listing 4.11. Random tests for `sortedDigits`

```
b) sortedDigits Random (3ms): Error Message: Test method Tests+Tests.b)
↪  sortedDigits Random threw exception:
  System.Exception: Falsifiable, after 5 tests (0 shrinks) (StdGen (
  ↪  867569077, 297439050)):
  Original:
  33N
  with exception:
  Microsoft.VisualStudio.TestTools.UnitTesting.AssertFailedException:
  ↪  Assert.AreEqual failed. Expected:<True>. Actual:<False>.
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.
        ↪  ThrowAssertFailed(String assertionName, String message) in /
        ↪  _/src/TestFramework/TestFramework/Assertions/Assert.cs:line
        ↪  60
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[
        ↪  T](T expected, T actual, IEqualityComparer`1 comparer,
        ↪  String message, Object[] parameters) in /_/src/TestFramework
        ↪  /TestFramework/Assertions/Assert.AreEqual.cs:line 201
      at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T]
      ↪  (T expected, T actual) in /_/src/TestFramework/TestFramework/
      ↪  Assertions/Assert.AreEqual.cs:line 34
        at Tests.b) sortedDigits Random@90.Invoke(Nat n) in path\to\
        ↪  Tests.fs:line 90
        at FsCheck.Testable.evaluate[a,b](FSharpFunc`2 body, a a)

  Stack Trace:
      at Microsoft.FSharp.Core.PrintfModule.PrintFormatToStringThenFail@
      ↪  1448.Invoke(String message)
      at <StartupCode$FsCheck>.$Runner.get_throwingRunner@382.FsCheck.
      ↪  IRunner.OnFinished(String name, TestResult testResult)
        at FsCheck.Runner.runner[a](Config config, a prop)
        at FsCheck.Runner.check[a](Config config, a p)
        at FsCheck.Check.One[Testable](Config config, Testable property)
        at Tests.Tests.b) sortedDigits Random() in path\to\Tests.fs:line
        ↪   89
        at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
        ↪   arguments, Signature sig, Boolean isConstructor)
        at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object
        ↪  obj, BindingFlags invokeAttr)
```

Listing 4.12. Output of random tests for `sortedDigits`

Tested expression	`b) sortedDigits Random` / `Original: 33N` } `sortedDigits 33N`
Expected result	`Expected:<True>` $\Longrightarrow$ `True` (actually `true`)
Actual result	`Actual:<False>` $\Longrightarrow$ `False` (actually `false`)

Table 4.4. Title

Key Information Random Tests — `sortedDigits`

4.2. Information Gaps in Error Outputs

One thing error messages lack is information about the context of a failed test. Consider an implementation of a singly linked list with no duplicates. The linked list contains the attributes: pointer to both the first and last node of the list and the size of list. Each node contains a value and a pointer to the next element.

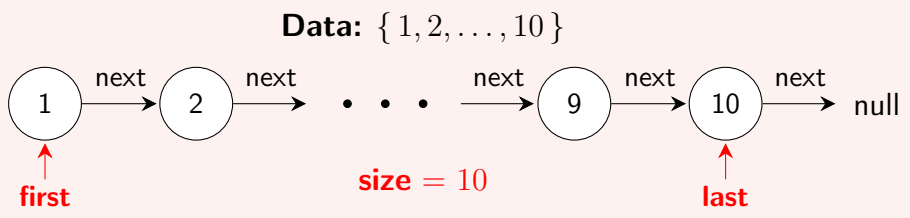

Figure 4.1. Illustration singly linked list

Furthermore consider the implemented methods:

- `pushFront: 'a -> LinkedList -> unit`
- `pushBack: 'a -> LinkedList -> unit`
- `insertAtIndex: 'a -> int -> LinkedList -> unit`
- `remove: 'a -> LinkedList -> unit`

These methods would need to be tested for an empty and non-empty list. As well as whether combinations of method application do not break the list, namely: Inserting and element and then removing it, removing elements until the list is empty and so on.

TODO

Illustrate how tests for this example might leave gaps because it misses the information of what was actually tested. How many elements were inserting before the removal of an element somehow failed? Introduce idea of a **history**, so that every error message comes with a history of the executed operations before something failed.

4.3. Challenges for Novices in Deriving Test Information

TODO

Explain why deciphering test expectations is hard for beginners. Elaborate that the information is spread across the entire error message and one has to derive essential information.

5. Assertify: A Library for Improving Error Messages

Assertify describes a library of various methods and classes to improve error messages during testing.

5.1. Objectives and Design of Assertify

Assertify incorporates functionalities to enhance the debugging and testing experience by providing clearer, more informative error messages. Traditional assertion failures often yield cryptic outputs, making it difficult to identify the root cause of an issue. Assertify addresses this by offering detailed, structured feedback, including expression simplifications, expected vs. actual values, and reduction history. The design principles behind Assertify focus on:

(1) **Clarity** – Displaying test failures in a structured and easily readable format.

(2) **Context** – Providing additional context through expression reductions and history tracking.

(3) **Usability** – Ensuring that error messages are helpful for students and developers alike.

(4) **Flexibility** – Supporting different levels of verbosity for various debugging needs.

By integrating these principles, Assertify streamlines the debugging process and improves the efficiency of error resolution.

5.2. Implementation Details and Wrappers

Assertify is implemented as a wrapper around standard test assertions, enriching the output with structured details. The core mechanism involves capturing the tested expressions, simplifying them where possible, and formatting the results into a well-structured report. Key implementation details include:

- **Expression Reduction:** Assertify simplifies complex expressions, stripping unnecessary details while preserving meaningful structure.
- **History Tracking:** The system logs successfully evaluated expressions leading up to a failure, providing insight into the progression of computations. This is especially helpful if a succession of operations are executed on the

same data structure and said data structure is correctly implemented for the majority of operations and only fails when a specific edge case occurs (e.g. an (almost) full linked list encounters a out-of-bounds index error). Then the history of operations might be helpful for novices to recreate the error because the order of operations is crutial to whether or not a operation fails.

- **Verbose and Compact Modes:** Users can toggle between detailed explanations and minimalistic outputs.
- **Enhanced Formatting:** Utilizing structured markers and color-coded elements, Assertify highlights critical differences between expected and actual results.

By wrapping around existing test frameworks, Assertify seamlessly integrates into existing workflows, enhancing error visibility without requiring significant changes to test code.

5.3. Error Message Improvements (Before and After Comparisons)

Traditional assertion failures often return extremely detailed messages that require additional effort to decipher. Assertify enhances this by providing detailed comparisons. The enhanced output not only shows expected vs. actual values but also presents the exact expression tested, making debugging significantly easier.

5.3.1. Example 1 — `digitSum`

TODO

Apparently there is something to do.

Example Tests — `digitSum`

TODO

(1) Compare old and new test results (old: 4.4.)

```
-------------------- TESTED EXPRESSION -----------------------
Tested/Failed expression:   StudentSubmission.digitSum 123N
Simplified expression:      digitSum 123N
------------------- EXPECTED AND ACTUAL ----------------------
Expected result:            6N
Actual result:              7N
```

Listing 5.1. Assertify output of examples tests for `digitSum`

Random Tests — `digitSum`

TODO

(1) Compare old and new test results (old: 4.6.)

```
-------------------- TESTED EXPRESSION -----------------------
Tested/Failed expression:   StudentSubmission.digitSum n
Simplified expression:      digitSum 0N
------------------- EXPECTED AND ACTUAL ----------------------
Expected result:            0N
Actual result:              1N
```

Listing 5.2. Assertify output of examples tests for `digitSum`

5.3.2. Example 2 — `sortedDigits`

TODO

Apparently there is something to do.

Example Tests — `sortedDigits`

TODO

(1) Compare old and new test results (old: 4.10.)

```
-------------------- TESTED EXPRESSION -------------------------
Tested/Failed expression:   StudentSubmission.sortedDigits 1111N
Simplified expression:      sortedDigits 1111N
------------------- EXPECTED AND ACTUAL ------------------------
Expected result:            true
Actual result:              false
```

Listing 5.3. Assertify output of examples tests for `sortedDigits`

Random Tests — `sortedDigits`

TODO

(1) Compare old and new test results (old: 4.12.)

```
-------------------- TESTED EXPRESSION -----------------------
Tested/Failed expression:   StudentSubmission.sortedDigits n
Simplified expression:      sortedDigits 33N
------------------- EXPECTED AND ACTUAL ---------------------
Expected result:            true
Actual result:              false
```

Listing 5.4. Assertify output of examples tests for `sortedDigits`

5.4. Usability and Design Considerations

Assertify is designed with a strong focus on usability, particularly for students learning programming and debugging. Key considerations include:

- **Readable Error Reports:** Formatting failures in a structured, natural-language format ensures accessibility.
- **Minimal Intrusion:** Assertify seamlessly integrates with existing test frameworks without requiring major changes.

- **Customizability:** Users can adjust verbosity levels and enable/disable history tracking based on their needs.
- **Color Coding and Annotations:** Highlighting key differences improves readability and makes debugging more intuitive.

These design choices collectively improve the debugging experience, reducing frustration and helping developers quickly understand and fix issues.

6. TODO: Further Methodology (unclear what comes next)

TODO

(1) VS Code Extension

(2) ExClaim Embedding

7. Evaluation and Results

TODO

Apparently there is something to do.

7.1. Evaluation Methodology

TODO

Explain how you assessed the effectiveness of Assertify.

7.2. Metrics for Success

TODO

Describe the criteria used to measure improvements.

7.3. Quantitative Analysis

TODO

Present the data and statistical results.

7.4. Qualitative Insights

TODO

Share insights from student feedback or observations.

7.5. Comparison to Previous Results

TODO

Highlight improvements compared to the baseline semester data.

8. Conclusion

TODO

Apparently there is something to do.

8.1. Summary of Contributions

TODO

Recap your key contributions.

8.2. Key Findings

TODO

Highlight the most important results and insights.

8.3. Interpretation of Results

TODO

Analyze what the results reveal about error messages and learning.

8.4. Strengths and Limitations of Approach

TODO

Identify the strengths and potential weaknesses of the approach.

8.5. Recommendations for Future Development

TODO

Suggest possible extensions or refinements.

List of Figures

3.1. Build Stability Analysis 11
3.2. Submission Performance Analysis 12
3.3. Error Code Analysis 17

4.1. Illustration singly linked list 26

List of Listings

1.1. Example Ouput of `Assert.AreEqual` 1

4.1. Student submission for `digitSum` 19
4.2. Sample solution for `digitSum` 20
4.3. Example tests for `digitSum` 21
4.4. Output of examples tests for `digitSum` 21
4.5. Random tests for `digitSum` 22
4.6. Output of random tests for `digitSum` 22
4.7. Student submission for `sortedDigits` 23
4.8. Sample solution for `sortedDigits` 23
4.9. Example tests for `sortedDigits` 24
4.10. Output of examples tests for `sortedDigits` 24
4.11. Random tests for `sortedDigits` 25
4.12. Output of random tests for `sortedDigits` 25

5.1. Assertify output of examples tests for `digitSum` 31
5.2. Assertify output of examples tests for `digitSum` 31
5.3. Assertify output of examples tests for `sortedDigits` 32
5.4. Assertify output of examples tests for `sortedDigits` 32

List of Tables

3.1. The top 10 of `F#` error codes in all submissions (1st – 5th) 15
3.2. The top 10 of `F#` error codes in all submissions (6th – 10th) . . . 16

4.1. Key Information Example Tests — `digitSum` 21
4.2. Title 23
4.3. Title 24
4.4. Title 26

A. My Code

TODO

Put detailed code here.

B. My Ideas

TODO

Ideas??

Bibliography

[1] Titus Barik, Justin Smith, Kevin Lubick, Elisabeth Holmes, Jing Feng, Emerson Murphy-Hill, and Chris Parnin. "Do Developers Read Compiler Error Messages?" In: *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. 2017, pp. 575–585. DOI: 10.1109/ICSE.2017.59.

[2] Brett A. Becker, Paul Denny, Raymond Pettit, Durell Bouchard, et al. "Compiler Error Messages Considered Unhelpful: The Landscape of Text-Based Programming Error Message Research". In: *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education*. ITiCSE-WGR '19. Aberdeen, Scotland Uk: Association for Computing Machinery, 2019, pp. 177–210. ISBN: 9781450375672. DOI: 10.1145/3344429.3372508. URL: https://doi.org/10.1145/3344429.3372508.